

Work Trial Exercise: AI Medical Scheduling Agent

Overview

Build a voice-based scheduling agent that can book patient appointments at a medical facility, backed by a Flask API and a simple call-review dashboard.

This project is intentionally larger than what can be completed in two days. We are not expecting a finished, production-ready system. We are evaluating how you prioritize, what you choose to build first, how you handle ambiguity, and how clean and well-reasoned your work is under time pressure — not raw feature count. A small set of things done well will score higher than a large set of things done half-way.

Timeline

- You have **two calendar days** from whenever you start.
- Submit whenever you're done, or whenever the two days are up — whichever comes first. Partial submissions are expected and fine.
- Spend the first chunk of time planning and prioritizing before writing code. We'd rather see a short note on what you decided to skip and why than have you discover at the end you spent six hours on something low-priority.

What You're Building

1. Conversational flow (Vogent)

A flow-based caller agent that can schedule a patient appointment end-to-end over a phone call.

The agent should be able to:

- Identify whether the caller is a new or returning patient
- It should be smart enough to route patients according to their status of new vs returning
- Determine the reason for the visit (e.g. follow-up, new injury, fracture, general consult)
- Use the physician protocol document (provided separately) to match the patient's issue and appointment type to the right doctor and location
- Recognize when a doctor doesn't take a given appointment type or issue (e.g. a doctor who doesn't see new patients, or doesn't handle fractures), explain that to the caller in plain language, and redirect them to an appropriate alternative
- Check real availability and offer slots dynamically, if a preferred doctor has no openings, move to the next reasonable option without breaking the conversation
- Confirm the booking back to the caller before ending the call

2. Physician protocol document

A reference document (we will provide this separately) listing doctors, their specialties, locations, which appointment types/issues they do and don't accept, and any other constraints the agent needs to route correctly.

3. Flask backend

An API the Vogent flow calls into during the conversation. At minimum:

- **Patient lookup / creation** — find an existing patient or create a new one
- **Slot lookup** — given a doctor (or issue/appointment type), return available slots
- **Scheduling** — book an appointment into a given slot
- **Appointment lookup** — retrieve existing appointment(s) for a patient
- Anything else you find you need to support the flow cleanly (e.g. doctor/specialty lookup, cancellation/reschedule if you have time)

Design the schema and endpoints sensibly — this will be read as a signal of how you structure a real backend, not just whether it works.

4. Call review frontend

A simple frontend showing the calls made to the agent. Each call entry should show:

- Full transcript
- Booking status (e.g. scheduled, abandoned, redirected, failed)
- Patient information
- If successfully scheduled: appointment details (doctor, location, time)

This can be minimal, it's there to demonstrate the data is captured correctly and is reviewable, not to be a polished product.

Deployment

The backend and frontend should be deployed on an AWS EC2 instance, dockerized if possible (e.g. a Docker container for the backend and one for the frontend). We want to see this running somewhere we can hit, not just on your laptop.

Submission

Please include:

- Repo/code (Flask backend, Vogent flow export/config, frontend)
- A short README covering: what you built, what you deliberately skipped and why, and what you'd do next with more time
- A video showcasing the platform, as well as a brief overview of the code structure

We're looking for clear signal on speed, judgment, and code quality over completeness — don't burn your two days chasing 100% coverage of the protocol document. Let me know if you have any questions or need anything, good luck!